

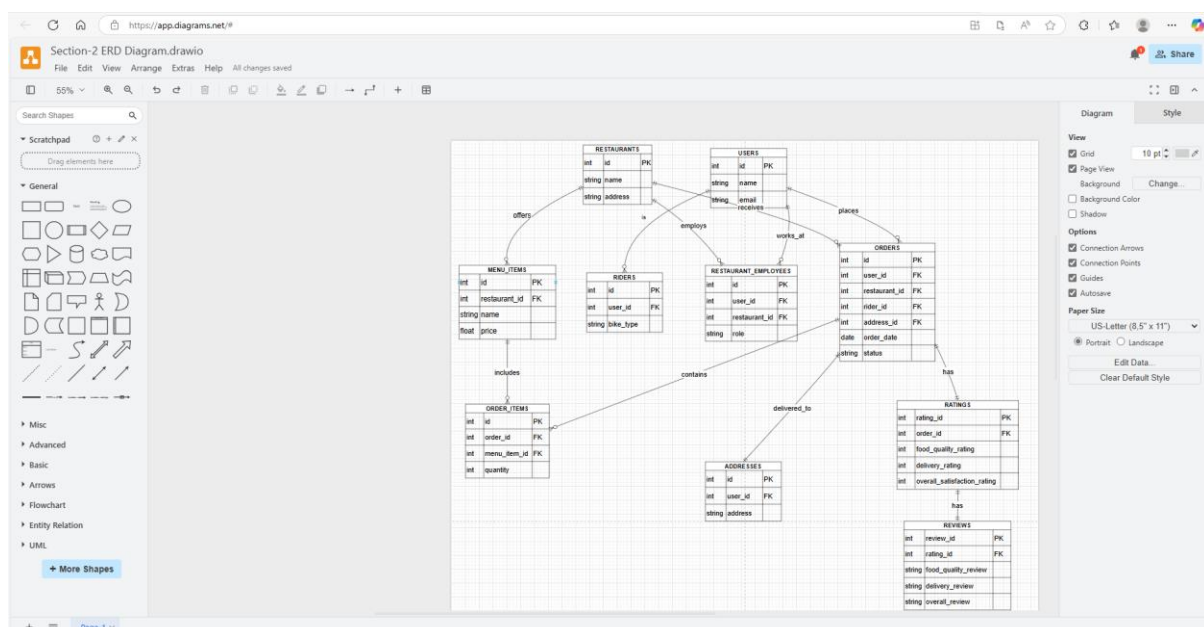
Section 2: Database Design Extension – Ratings and Reviews

ER Diagram

I created & updated Entity-Relationship Diagram includes two new entities to support this new functionality, I added two new tables to the database: RATINGS and REVIEWS. Each rating is linked to an order, and each review is linked to a rating. I used foreign key constraints to make sure these relationships are enforced. Here's how the structure looks in the ER diagram:

RATINGS table: stores numeric values for each rating category and is linked to the **ORDERS** table.

REVIEWS table: stores optional text reviews and is linked to the **RATINGS** table.



ER-Diagram: [ER Diagram.drawio - draw.io](https://app.diagrams.net/#Section-2%20ERD%20Diagram.drawio)

Third Normal Form (3NF)

I made sure the new tables follow the rules of Third Normal Form which is also known as 3NF here,

First Normal Form (1NF): All values are atomic (no repeating groups or arrays).

Second Normal Form (2NF): All non-primary key attributes depend on the whole primary key.

Third Normal Form (3NF): There are no transitive dependencies. All fields are directly related to the primary key and not to other non-key fields.

RATINGS Table

First Normal Form(1NF): All attributes are atomic (individual rating values).

Second Normal Form(2NF): All non-key attributes depend on the full primary key rating_id.

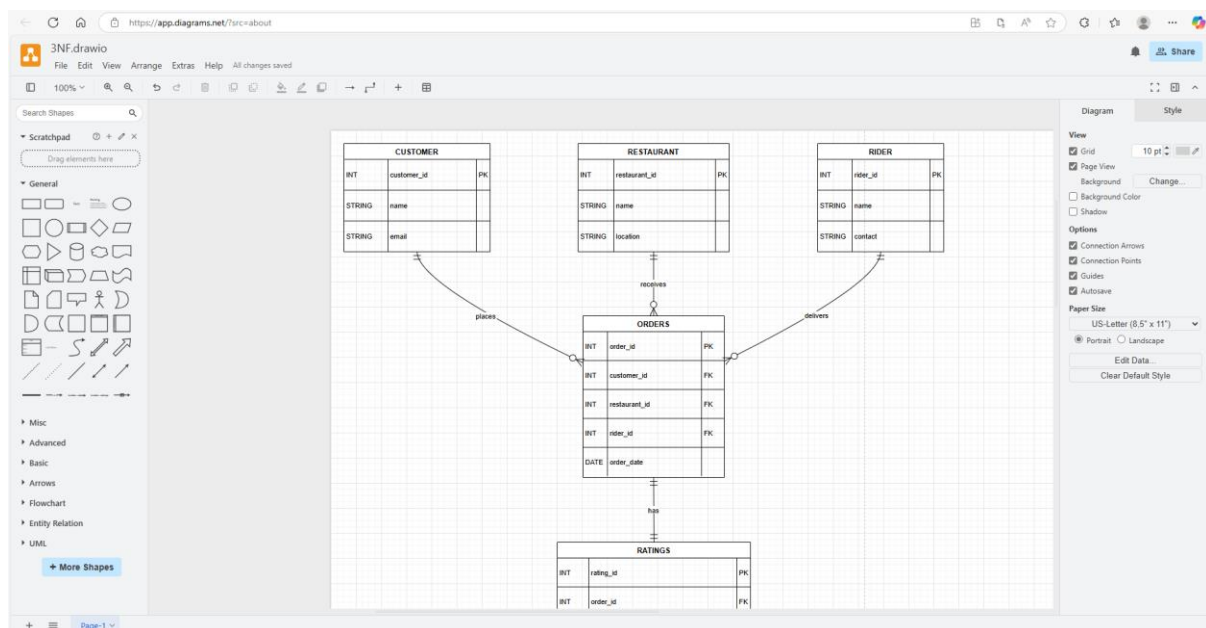
Third Normal Form(3NF): No transitive dependencies. Each rating attribute is directly dependent on the primary key.

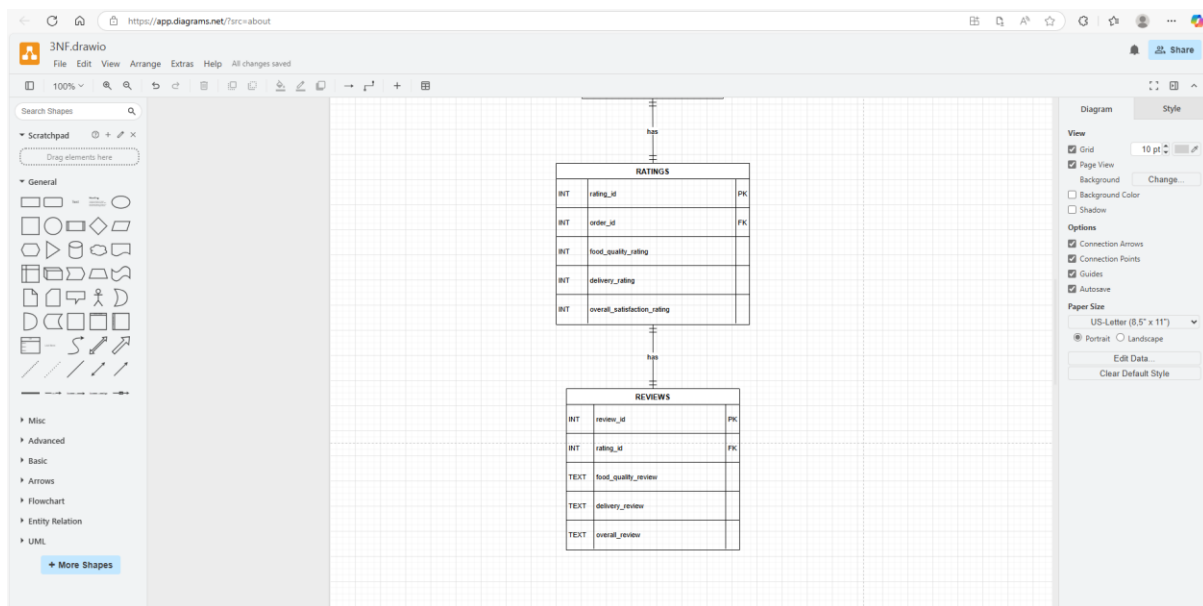
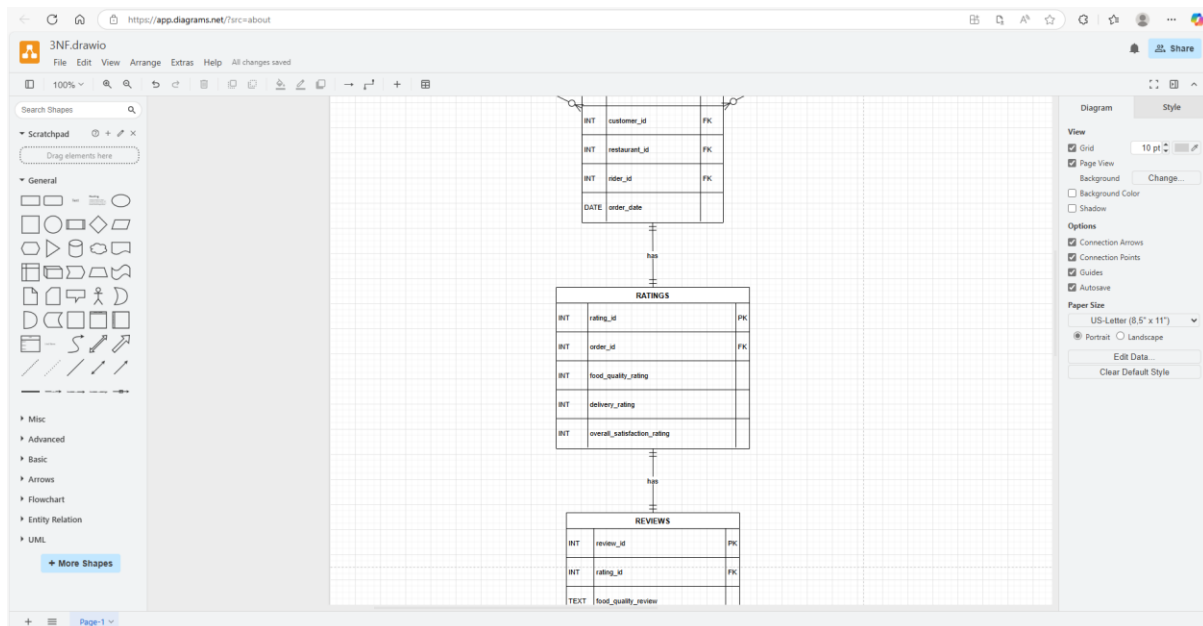
REVIEWS Table

First Normal Form(1NF): Each review field is atomic text.

Second Normal Form(2NF): All attributes depend fully on review_id or its foreign key rating_id.

Third Normal Form(3NF): No non-key attributes depend on each other; all depend on rating_id.





3NF LINK: [3NF.drawio - draw.io](https://app.diagrams.net?arc=about)

SQL Implementation

I already created two new tables in the putty-SSH for the Ratings, and Reviews for implementing and showing those tables on my database system here It is-

SELECT * FROM REVIEWS;

SELECT * FROM RATINGS;

Here is the Table of SQL query for REVIEWS & RATINGS-

DROP TABLE IF EXISTS REVIEWS;

DROP TABLE IF EXISTS RATINGS;

CREATE TABLE RATINGS (

rating_id INT AUTO_INCREMENT PRIMARY KEY,

order_id INT NOT NULL UNIQUE,

food_quality_rating INT CHECK (food_quality_rating BETWEEN 0 AND 5),

delivery_rating INT CHECK (delivery_rating BETWEEN 0 AND 5),

overall_satisfaction_rating INT CHECK (overall_satisfaction_rating BETWEEN 0 AND 5)

);

CREATE TABLE REVIEWS (

review_id INT AUTO_INCREMENT PRIMARY KEY,

rating_id INT NOT NULL UNIQUE,

food_quality_review TEXT,

delivery_review TEXT,

overall_review TEXT,

FOREIGN KEY (rating_id) REFERENCES RATINGS(rating_id) ON DELETE CASCADE

);

INSERT INTO RATINGS (order_id, food_quality_rating, delivery_rating,
overall_satisfaction_rating) VALUES

(100100, 5, 4, 5),

(100101, 3, 4, 3),

(100102, 4, 4, 4),

(100103, 5, 5, 5),

(100104, 3, 2, 3),

(100105, 4, 3, 4);

INSERT INTO REVIEWS (rating_id, food_quality_review, delivery_review, overall_review)
VALUES

(1, 'Delicious food', 'Quick delivery', 'Great experience'),

(2, 'Okay food', 'Average speed', 'Not bad overall'),

(3, 'Tasty and warm', 'Fast delivery', 'Very pleased');

INSERT INTO RATINGS (order_id, overall_satisfaction_rating) VALUES

(100106, 4),

(100107, 3),

(100108, 5),

(100109, 2),

(100110, 4),

(100111, 5);

INSERT INTO REVIEWS (rating_id, overall_review) VALUES

(7, 'Very satisfied'),

(8, 'Could be better'),

(9, 'Excellent service');

```
6 rows in set (0.000 sec)

MariaDB [40735842]: SELECT * FROM RATINGS;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near 'RATINGS' at line 1
MariaDB [40735842]: SELECT * FROM RATINGS;

+-----+
| rating_id | order_id | food_quality_rating | delivery_rating | overall_satisfaction_rating |
+-----+
| 1 | 1000001 | 5 | 4 | 5 |
| 2 | 1000002 | 4 | 3 | 4 |
| 3 | 1000003 | NULL | 3 | NULL |
| 4 | 1001001 | 5 | 4 | 5 |
| 5 | 1001011 | 3 | 4 | 3 |
| 6 | 1001021 | 4 | 4 | 4 |
| 7 | 1001031 | 5 | 5 | 5 |
| 8 | 1001041 | 3 | 2 | 3 |
| 9 | 1001051 | 4 | 3 | 4 |
| 10 | 1001061 | NULL | NULL | 4 |
| 11 | 1001071 | NULL | NULL | 3 |
| 12 | 1001081 | NULL | NULL | 5 |
| 13 | 1001091 | NULL | NULL | 2 |
| 14 | 1001101 | NULL | NULL | 4 |
| 15 | 1001111 | NULL | NULL | 5 |
+-----+
15 rows in set (0.000 sec)

MariaDB [40735842]: SELECT * FROM REVIEWS;

+-----+
| review_id | rating_id | food_quality_review | delivery_review | overall_review |
+-----+
| 1 | 1 | Delicious food | Quick delivery | Great experience |
| 2 | 2 | Okay food | Average speed | Not bad overall |
| 3 | 3 | Tasty and warm | Fast delivery | Very pleased |
| 4 | 7 | NULL | NULL | Very satisfied |
| 5 | 8 | NULL | NULL | Could be better |
| 6 | 9 | NULL | NULL | Excellent service |
+-----+
6 rows in set (0.000 sec)

MariaDB [40735842]: SELECT * FROM RATINGS;

+-----+
| rating_id | order_id | food_quality_rating | delivery_rating | overall_satisfaction_rating |
+-----+
| 1 | 1000001 | 5 | 4 | 5 |
| 2 | 1000002 | 4 | 3 | 4 |
| 3 | 1000003 | NULL | 3 | NULL |
| 4 | 1001001 | 5 | 4 | 5 |
| 5 | 1001011 | 3 | 4 | 3 |
| 6 | 1001021 | 4 | 4 | 4 |
| 7 | 1001031 | 5 | 5 | 5 |
| 8 | 1001041 | 3 | 2 | 3 |
| 9 | 1001051 | 4 | 3 | 4 |
| 10 | 1001061 | NULL | NULL | 4 |
| 11 | 1001071 | NULL | NULL | 3 |
| 12 | 1001081 | NULL | NULL | 5 |
| 13 | 1001091 | NULL | NULL | 2 |
| 14 | 1001101 | NULL | NULL | 4 |
| 15 | 1001111 | NULL | NULL | 5 |
+-----+
15 rows in set (0.000 sec)

MariaDB [40735842]:
```

This SQL query for the No Rating Table -

DROP TABLE IF EXISTS REVIEWS;

DROP TABLE IF EXISTS RATINGS;

CREATE TABLE RATINGS (

rating_id INT AUTO_INCREMENT PRIMARY KEY,

order_id INT NOT NULL UNIQUE,

food_quality_rating INT CHECK (food_quality_rating BETWEEN 0 AND 5),

delivery_rating INT CHECK (delivery_rating BETWEEN 0 AND 5),

overall_satisfaction_rating INT CHECK (overall_satisfaction_rating BETWEEN 0 AND 5),

FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE CASCADE

);

CREATE TABLE REVIEWS (

review_id INT AUTO_INCREMENT PRIMARY KEY,

rating_id INT NOT NULL UNIQUE,

food_quality_review TEXT,

```
delivery_review TEXT,  
overall_review TEXT,  
FOREIGN KEY (rating_id) REFERENCES RATINGS(rating_id) ON DELETE CASCADE  
);
```

```
INSERT INTO RATINGS (order_id, food_quality_rating, delivery_rating,  
overall_satisfaction_rating)  
VALUES (100001, 5, 4, 5);
```

```
INSERT INTO REVIEWS (rating_id, food_quality_review, delivery_review, overall_review)  
VALUES (1, 'Fresh and tasty', 'Delivered early', 'Excellent experience');
```

```
INSERT INTO RATINGS (order_id, food_quality_rating, delivery_rating,  
overall_satisfaction_rating)  
VALUES (100002, 4, 3, 4);
```

```
INSERT INTO RATINGS (order_id, overall_satisfaction_rating)  
VALUES (100003, 3);
```

```
SELECT  
    COALESCE(ROUND(AVG(r.food_quality_rating), 1), 'No rating available') AS  
avg_food_quality,  
    COALESCE(ROUND(AVG(r.delivery_rating), 1), 'No rating available') AS avg_delivery,  
    COALESCE(ROUND(AVG(r.overall_satisfaction_rating), 1), 'No rating available') AS  
avg_overall  
FROM orders o  
LEFT JOIN RATINGS r ON o.id = r.order_id  
WHERE o.restaurant_id = 123  
AND o.order_date >= NOW() - INTERVAL 4 WEEK;
```

SELECT

COALESCE(ROUND(AVG(r.delivery_rating), 1), 'No rating available') AS
avg_rider_delivery_rating

FROM orders o

LEFT JOIN RATINGS r ON o.id = r.order_id

WHERE o.rider_id = 456

AND o.order_date >= NOW() - INTERVAL 10 DAY;

```
40735842@ushe-db-64-01:
mysql
-- Query A: Restaurant rating summary
-- Replace '456' with an actual restaurant_id that exists in your data
SELECT
CASE
  WHEN COUNT(r.rating_id) = 0 THEN 'No rating available'
  ELSE CONCAT('Food Quality: ', ROUND(AVG(r.food_quality_rating), 1),
    ', Delivery: ', ROUND(AVG(r.delivery_rating), 1),
    ', Overall: ', ROUND(AVG(r.overall_satisfaction_rating), 1))
END AS restaurant_rating_summary
FROM orders o
LEFT JOIN RATINGS r ON o.id = r.order_id
WHERE o.restaurant_id = 123
AND o.order_date >= NOW() - INTERVAL 4 WEEK;

-- Query B: Average delivery rating for a rider in the last 10 days
-- Replace '456' with an actual rider_id that exists in your data
SELECT
CASE
  WHEN COUNT(r.rating_id) = 0 THEN 'No rating available'
  ELSE CONCAT('Delivery Rating: ', ROUND(AVG(r.delivery_rating), 1))
END AS rider_rating_summary
FROM orders o
LEFT JOIN RATINGS r ON o.id = r.order_id
WHERE o.rider_id = 456
AND o.order_date >= NOW() - INTERVAL 10 DAY;

+-----+
| restaurant_rating_summary |
+-----+
| No rating available       |
+-----+
1 row in set (0.320 sec)

MariaDB [40735842]>
MariaDB [40735842]> -- Query B: Average delivery rating for a rider in the last
10 days
MariaDB [40735842]> -- Replace '456' with an actual rider_id that exists in your
data
MariaDB [40735842]> SELECT
CASE
  WHEN COUNT(r.rating_id) = 0 THEN 'No rating available'
  ELSE CONCAT('Delivery Rating: ', ROUND(AVG(r.delivery_rating), 1))
END AS rider_rating_summary
FROM orders o
LEFT JOIN RATINGS r ON o.id = r.order_id
WHERE o.rider_id = 456
AND o.order_date >= NOW() - INTERVAL 10 DAY;

+-----+
| rider_rating_summary |
+-----+
| No rating available   |
+-----+
1 row in set (0.003 sec)

MariaDB [40735842]>
```